

Linear Regression: From First Principles

Why Do We Need Machine Learning?

Imagine you own a real estate company. Every day, customers ask the same question: "How much is my house worth?" When you have only a few houses, estimating the price is easy. You can compare the new house with previous sales and make a reasonable guess. Now imagine your company grows. You now have information about 100,000 houses. Each house has details such as:

- House size
- Number of bedrooms
- Number of bathrooms
- Location
- Age of the house
- Selling price

A customer walks in and asks you to estimate the price of a new house. Could you manually compare that house with all 100,000 houses?

Probably not. Even if you could, it would take a long time and people would still make mistakes. Clearly, we need a better solution.

Learning Instead of Memorizing

Instead of searching through thousands of old records every time, wouldn't it be better if a computer could study the old data once, understand the pattern, and then make predictions automatically?

That is exactly what Machine Learning does. Machine Learning does not memorize every example. Instead, it learns the relationship between the input and the output so it can make predictions for data it has never seen before.

Imagine teaching a child what a dog looks like. You show the child many different dogs. After seeing enough examples, the child begins to notice common characteristics. The child is no longer memorizing individual dogs. Instead, the child has learned the pattern that makes a dog a dog. Machine Learning works in the same way. It learns patterns from data.

Machine Learning is the process of teaching a computer to learn patterns from existing data so it can make predictions or decisions on new, unseen data without being explicitly programmed for every situation.

Why Are We Learning Linear Regression?

Machine Learning contains many different algorithms. Some algorithms classify emails as spam or not spam. Some recognize faces. Some recommend movies. Some predict numbers. Whenever our goal is to predict a continuous numerical value such as:

- House price

- Salary
- Temperature
- Sales
- Exam score

one of the simplest and most important algorithms is Linear Regression. This book focuses on understanding Linear Regression from first principles.

What is Linear Regression?

The next question is: **How does Linear Regression learn those patterns?**

Imagine a teacher writes the following information on the board.

Study Hours	Exam Score
1	40
2	50
3	60
4	70
5	80

Without doing any calculations, what do you notice?

You will probably say,

"The more a student studies, the higher the exam score."

Notice something important. You didn't memorize the table. Instead, your brain discovered a **pattern**. That is exactly what Linear Regression tries to do. It looks at existing data and tries to discover the relationship between the input and the output.

Definition Linear Regression is a Machine Learning algorithm that learns the relationship between one or more input variables and an output variable by fitting the best possible straight line through the data. That definition sounds technical. Let's make it much simpler.

Imagine I ask you,

"If a student studies for 6 hours, what score might they get?"

The table only contains information up to 5 hours. So how can we answer?

Our brain naturally extends the pattern. We might say,

"Probably around 90."

Did we know the exact answer?

No. We simply used the pattern to make a prediction. That is exactly what Linear Regression does.

Why is it called "Linear"?

The word **Linear** simply means **Straight Line**. Imagine plotting the study data on a graph. The points would look something like this.

The points would rise from left to right: as study hours increase, the exam score also increases. This upward pattern is what the line tries to capture.

Notice that the points almost form a straight line. Linear Regression says,

"Instead of remembering every point, I will draw one straight line that represents the overall pattern."

That straight line is called the **Best Fit Line**.

Why Do We Need the Best Fit Line?

Imagine another student comes to you. They studied for **3.5 hours**. The table doesn't contain that value. Should we guess?

No. Instead, we use the Best Fit Line to estimate the score. The line helps us make predictions for values we have never seen before. This is one of the biggest advantages of Machine Learning. It can make predictions for **new data**, not just data it has already seen.

What Does "Best Fit" Mean?

Imagine four friends standing in different places. You want to stand somewhere that keeps you reasonably close to everyone. You probably won't stand exactly beside one person. Instead, you'll choose a position that is the best compromise. The Best Fit Line works in the same way. It usually does **not** pass through every data point. Instead, it tries to stay as close as possible to all of them. That is why it is called the **Best Fit Line**.

Does Linear Regression Memorize the Data?

No. This is one of the biggest misconceptions. Suppose our training data contains these house sizes. 1000 sq ft 1500 sq ft 2000 sq ft 2500 sq ft Now someone asks,

"What about a house with 1800 sq ft?"

The model has never seen that exact value before. It cannot memorize something that doesn't exist. Instead, it uses the relationship it learned to estimate the answer. This is why we say Machine Learning **learns**, not **memorizes**.

Real-Life Examples of Linear Regression

Linear Regression is used whenever we want to predict a continuous numerical value. Some common examples include:

- Predicting house prices.
- Predicting employee salaries.
- Predicting sales for next month.
- Predicting electricity consumption.
- Predicting exam scores.
- Predicting the temperature tomorrow.

Notice something. Every prediction is a **number**. That is where Linear Regression is commonly used.

The Equation of a Straight Line

A new question arises.

How does a computer describe that line?

Just like humans, a computer uses an equation.

Every Straight Line Has an Equation

Long before Machine Learning existed, mathematics already had an equation for every straight line. That equation is:

$$y = mx + c$$

Machine Learning did not invent a new equation. It simply uses this equation to learn patterns from data. Before we understand Linear Regression, we first need to understand this equation.

Understanding the Equation

The equation looks very small.

$$y = mx + c$$

But every symbol has a job. Let's understand each one.

What is x?

Think of **x** as the information you already know. For example, suppose you want to predict a student's exam score. What information do you already know?

Maybe you know:

- Study Hours = 5

Here,

$$x = 5$$

Another example: suppose you want to predict a house price. You already know: House Size = 2000 sq ft. Then,

$$x = 2000$$

A simple way to remember it is:

x is the input we already know.

What is y?

Now suppose I ask,

"A student studied for 5 hours. What will be the exam score?"

The exam score is still unknown. That unknown value is **y**. Another example: suppose a house has an area of 2000 sq ft. You want to predict its price. The house price is **y**. A simple way to remember it is:

y is the value we want to predict.

What is m?

This is the most important part of the equation. The value **m** is called the **Slope**. But what does "Slope" actually mean?

Imagine two roads. Road A is almost flat. Road B is very steep. Which road has the larger slope?

Obviously, Road B. The slope simply tells us

How steep the line is.

Another Way to Understand the Slope

Suppose

$$y = 2x + 5$$

The slope is

$$m = 2$$

This means: Whenever x increases by **1**, y increases by **2**. Let's test it.

x	y
1	7
2	9
3	11
4	13

Notice something. Every time x increases by 1, y increases by 2. That is exactly what the slope tells us.

Now imagine another equation.

$$y = 10x + 5$$

Now the slope is

10. Every time x increases by 1,

y increases by 10. This line becomes much steeper. So the slope controls

How quickly y changes when x changes.

What is c ?

The value c is called the **Intercept**. Instead of giving you the definition, let's discover it. Suppose

$$y = 2x + 5$$

Now ask yourself, "What happens if x becomes zero?" Let's calculate.

$$y = 2(0) + 5$$

$$y = 5$$

Interesting. When x becomes zero, the line still has a value. That value is called the **Intercept**. It is simply

The value of y when $x = 0$.

What Does the Intercept Do?

Imagine you have a ruler lying on a table. Now lift the entire ruler upward. Did the ruler become steeper?

No. It simply moved upward. Now place it back. This time, push it downward. Again, the ruler did not become steeper. It only changed its position. That is exactly what the intercept does. It moves the entire line

- Up
- Down

without changing its slope.

Putting Everything Together

Let's look at the equation again.

$$y = mx + c$$

Now every symbol has a meaning.

- $x \rightarrow$ Input
- $y \rightarrow$ Output
- $m \rightarrow$ Slope (how fast y changes)
- $c \rightarrow$ Intercept (starting point of the line)

One Complete Example

Suppose the equation is

$$y = 3x + 2$$

Let's predict some values.

x	y = 3x + 2
1	5
2	8
3	11
4	14

The equation keeps generating new predictions. Notice something. We didn't memorize these answers. We simply used the equation. This is exactly what Linear Regression will do.

How Does This Connect to Machine Learning?

Everything we learned in this topic already existed in mathematics. Machine Learning simply takes this equation and asks a new question.

How can a computer automatically find the best values of m and c?

Instead of manually choosing them, the computer will learn them from data. That is the entire idea behind Linear Regression.

The equation

$$y = mx + c$$

is the mathematical equation of a straight line. It uses the slope (m) and intercept (c) to describe the relationship between an input and an output.

From the Straight-Line Equation to Machine Learning Notation

Open any Machine Learning book. Instead of writing

$$y = mx + c$$

it writes

$$\hat{y} = \theta_0 + \theta_1 x$$

At first, this looks like a completely different equation. It isn't. It is exactly the same equation. Only the names have changed.

Why Did Machine Learning Change the Symbols?

Imagine your name is David. Now you move to another country. People start calling you "Dave." Did you become a different person?

No. Only the name changed. The same thing happened here. Mathematics calls it

$$y = mx + c$$

Machine Learning calls it

$$\hat{y} = \theta_0 + \theta_1 x$$

The equation didn't change. Only the symbols changed.

Let's Compare Them

School Mathematics	Machine Learning
y	$\hat{y}$
m	θ_1
c	θ_0
x	x

Notice something. Only three symbols changed. The idea stayed exactly the same.

Why Did y Become $\hat{y}$?

This is one of the most important changes. In mathematics, y usually means "The answer." In Machine Learning, we have two answers. The first answer is **The real answer**. The second answer is **The model's guess**. If we use the same symbol for both, it becomes confusing.

So Machine Learning introduces a new symbol.

$$\hat{y}$$

(read as "y-hat") The little hat simply means

This is a prediction, not the real answer.

Example

Suppose a house actually sells for \$320,000. That is

$$y = 320000$$

Now suppose our model predicts \$315,000. That becomes

$$\hat{y} = 315000$$

Notice something. There are now two values. Actual value is y , and predicted value is $\hat{y}$. Keeping them separate helps us calculate the error later.

Why Did c Become θ_0 ?

Remember, c was the intercept. It moved the line up and down. Machine Learning still needs an intercept. Instead of calling it c , it calls it

$$\theta_0$$

It does exactly the same job. Nothing changed except the name.

Example

Suppose

$$\hat{y} = 10 + 2x$$

Here, $\theta_0 = 10$. If x becomes zero,

$$\hat{y} = 10$$

Exactly like the intercept we learned before.

Why Did m Become θ_1 ?

Remember, m controlled the slope. Machine Learning still needs a slope. Instead of calling it m , it calls it

$$\theta_1$$

Again, nothing changed. Only the name changed.

Example

Suppose

$$\hat{y} = 5 + 3x$$

Here, $\theta_1 = 3$. That means every time x increases by 1, the prediction increases by 3. Exactly like the slope.

Why Do We Use θ Instead of m ?

This question usually comes later, but it's easier to answer now. Simple Linear Regression has only one feature. So one slope is enough. Later, we'll learn Multiple Linear Regression.

Suppose we have these features: House Size Bedrooms Bathrooms Age Now we need four different slopes. Writing m_1 m_2 m_3 m_4 is possible, but Machine Learning traditionally uses θ_1 , θ_2 , θ_3 , θ_4 for all model parameters. This keeps the notation consistent across many algorithms.

Our Final Equation

So from now on, whenever you see

$$\hat{y} = \theta_0 + \theta_1 x$$

you should immediately think

|"This is just $y = mx + c$ written in Machine Learning language."

Nothing more. Nothing less.

Suppose

$$\theta_0 = 2$$

$$\theta_1 = 4$$

The equation becomes

$$\hat{y} = 2 + 4x$$

Now predict the value when $x = 5$. First multiply $4 \times 5 = 20$, then add the intercept: $20 + 2 = 22$. So,

$$\hat{y} = 22$$

Very simple. The equation is just making predictions.

But There Is a Problem

Suppose the actual value was 20. Our model predicted 22. Was that prediction good?

Maybe. Maybe not. We need a way to measure **how wrong** our prediction is. That leads us to the next idea. **Error**.

The equation

$$\hat{y} = \theta_0 + \theta_1 x$$

is the Machine Learning version of the straight-line equation. It is used to predict the output based on the input.

- θ_0 is the intercept.
- θ_1 is the slope.
- $\hat{y}$ is the predicted value.

Prediction, Error and Cost Function

The computer now knows **how to make predictions**. But making predictions is only half the job. Now imagine I ask you a simple question.

"Is your model good?"

How would you answer?

You can't just say,

"It made a prediction."

Every model can make predictions. The real question is,

"How close are those predictions to the actual answers?"

To answer that, we need to understand two new ideas:

- Error
- Cost Function

Making a Prediction

Suppose our model is

$$\hat{y} = x$$

Now let's predict three values.

x	Actual y	Predicted $\hat{y}$
1	1	1
2	2	2
3	3	3

Every prediction is correct. This looks like a very good model. Now let's change the model. Suppose

$$\hat{y} = 2x$$

Now the predictions become

x	Actual y	Predicted $\hat{y}$
1	1	2
2	2	4
3	3	6

Now the predictions are no longer correct. But how wrong are they?

What is Error?

Imagine you throw a dart at a dartboard. If the dart lands exactly in the center, your error is zero. If it lands far away, your error is larger. Linear Regression works the same way. The prediction is like throwing the dart. The actual value is the center of the dartboard. The distance between them is called the **Error**.

Error is the difference between the actual value and the predicted value. The formula is

$$\text{Error} = y - \hat{y}$$

Some books write

$$\text{Error} = \hat{y} - y$$

Both are correct. Later we square the error, so the final Cost Function remains the same.

Using our second model,

Actual	Predicted	Error
1	2	-1
2	4	-2
3	6	-3

Notice something. Every prediction has its own error. So now I ask you,

"Is the model good?"

You still cannot answer. Why?

Because you have **three different errors**. Which one should you use?

The first?

The second?

The third?

Clearly, looking at one error is not enough.

Why Do We Need the Cost Function?

Imagine your teacher gives you a Mathematics exam with 100 questions. After the exam, you ask,

"How did I perform?"

Would your teacher reply,

"Question 17 was wrong."

Of course not. Your teacher looks at **all 100 questions** and gives you one final score. That final score tells you how well you performed overall. The Cost Function does exactly the same thing. Instead of looking at one prediction, it looks at **every prediction** and produces one number. That one number tells us

How good or bad the entire model is.

The Cost Function is a single number that measures how wrong the entire model is. Easy way to remember:

- Error → One prediction.
- Cost → The entire model.

Can We Simply Add the Errors?

Let's try. Suppose our errors are +5 -5 Now add them.

$$5 + (-5) = 0$$

According to this, our model made no mistakes. But is that true?

No. The model made two large mistakes. They simply cancelled each other. So adding the errors doesn't work. We need another idea.

Why Do We Square the Errors?

Instead of adding

+5 and -5, let's square them first.

$$(+5)^2 = 25$$

$$(-5)^2 = 25$$

Now add them.

$$25 + 25 = 50$$

Perfect. The errors can no longer cancel each other. There is another advantage. Suppose two models make these errors. Model A has errors 1, 2, 3. Model B has errors 1, 2, 10. Without squaring, 10 is only a little bigger than 3. After squaring, After squaring, Model A has 1, 4, 9. Model B has 1, 4, 100. Now the large mistake becomes much more noticeable. This is exactly what we want. A model making huge mistakes should receive a much larger penalty.

The Cost Function Formula

Now that we understand the idea, the formula becomes easy.

$$J(\theta) = \frac{1}{2m} \sum (\hat{y} - y)^2$$

Don't panic. Let's understand it one symbol at a time.

What is J?

J is simply the name of the Cost Function. Whenever you see

$$J(\theta)$$

think

Overall error of my model.

Nothing more.

What does Σ mean?

The symbol

$$\Sigma$$

is called **Sigma**. It simply means

Add everything together.

If there are 100 prediction errors, Sigma tells us to add all 100 squared errors.

Why Divide by m?

Suppose

Model A has

10 data points. Model B has 10,000 data points. If we only added the errors, Model B would almost always have a much larger total, simply because it has more data. That wouldn't be a fair comparison.

So we divide by **m**, the number of training examples. This gives us the **average error**.

Why Divide by 2?

This is one of the most common interview questions. The answer is surprisingly simple. Later, we will calculate the derivative of the Cost Function. The square creates a **2**. The **2** in the denominator cancels it. This makes the mathematics much cleaner. It does **not** change which model is best. It only makes the calculations easier.

Putting Everything Together

The Cost Function follows five simple steps.

The Cost Function process is: make predictions, calculate each error, square every error, add the squared errors, and divide by 2m.

The result is one number. That number tells us how good or bad the model is.

Why Do We Need Derivatives?

The Cost Function tells us one thing:

How wrong is my model?

Suppose we train three different models.

θ_1	Cost
0.5	0.583
1.0	0
2.0	2.333

Looking at the table, it is easy for us to say that

$\theta_1 = 1$ is the best value because it gives the smallest Cost.

But imagine you are a computer. You don't know that. You only know your current value of θ_1 . So the next question is,

How do I find a better θ_1 ?

Can We Just Keep Guessing?

One solution is to keep trying different values. For example, the computer could try $\theta_1 = 0.5$, calculate the Cost, then try $\theta_1 = 0.6$, then $\theta_1 = 0.7$, then $\theta_1 = 0.8$, and so on. Would this eventually find the answer?

Yes. Would it be efficient?

No. Now imagine trying 0.91, 0.912, 0.9125, 0.91254, 0.912541, and so on. There are infinitely many possible values. The computer needs a smarter method.

What Does the Computer Actually Need?

Think about driving a car. Suppose someone tells you,

"You are 20 miles away from your destination."

That's useful. But is it enough?

No. You immediately ask another question.

Which direction should I drive?

Knowing the distance is not enough. You also need the direction. The Cost Function tells us **how far we are from the best answer**. The derivative tells us **which direction to move to get closer**. This is why we need derivatives.

So What Is a Derivative?

Most textbooks say,

"A derivative is the instantaneous rate of change."

While that definition is correct, it is difficult to understand the first time you see it. Let's understand it using a simple example.

Consider the Function

$$f(x) = x^2$$

You may already know that

$$f'(x) = 2x$$

Many beginners think this means

x^2 becomes $2x$.

That is **not** what it means. The derivative is **not changing one equation into another**. Instead, it is answering a question.

If I change the input a tiny bit, how much will the output change?

Let's Use Numbers

Suppose

$$x = 1$$

The output is

$$1^2 = 1$$

Now increase x by only 0.01. The new input becomes

$$1.01$$

The new output becomes

$$1.01^2 = 1.0201$$

Let's compare them.

Input	Output
1.00	1.0000
1.01	1.0201

Now calculate the changes. Change in Input

$$1.01 - 1 = 0.01$$

Change in Output

$$1.0201 - 1 = 0.0201$$

Now divide them.

$$\frac{0.0201}{0.01} = 2.01$$

That value is approximately 2. This is why the derivative at $x = 1$ is 2.

What Does That "2" Mean?

It simply means

Near $x = 1$, every tiny increase in the input causes the output to increase about twice as much.

Notice something important. The derivative did not calculate the output. The original equation already does that. The derivative tells us **how fast the output is changing**.

Understanding the Sign

The sign of the derivative tells us the direction.

Positive Derivative

Suppose the derivative is

+5. That means when the input increases, the output also increases. As the input increases, the output also increases.

Negative Derivative

Suppose the derivative is

-5. That means when the input increases, the output decreases. As the input increases, the output decreases.

Zero Derivative

Suppose the derivative is

0. That means

changing the input slightly no longer changes the output. The graph has become flat.

Connecting This to Linear Regression

Forget x^2 for a moment. Now think about the Cost Function. Instead of asking

"If x changes, what happens?"

we ask

If θ_1 changes slightly, what happens to the Cost?

That is exactly what the derivative tells us. Suppose the derivative of the Cost Function is -3. That means

Increasing θ_1 will decrease the Cost.

Now suppose the derivative is +4. That means

Increasing θ_1 will increase the Cost.

So instead, we should decrease θ_1 . Notice something amazing. The derivative never tells us

"The correct value is $\theta_1 = 1$."

Instead, it simply says

"Move left."

or

"Move right."

That is enough. If we keep moving in the correct direction, we will eventually reach the best value.

Imagine Climbing Down a Mountain

Imagine you are standing on a mountain wearing a blindfold. Your goal is to reach the bottom. You cannot see anything. So how do you know where to walk?

You take one small step. If the ground goes downward, you continue walking. If the ground goes upward, you turn around. You repeat this over and over until the ground becomes flat. The mountain is the **Cost Function**. The bottom of the mountain is the **minimum Cost**. The derivative tells you

Which direction goes downhill.

A derivative measures **how much the output changes when the input changes by a very small amount**. In Linear Regression, the derivative tells us **how the Cost changes when the model parameters (θ_0 and θ_1) change**.

Gradient Descent

The derivative tells us:

"Which direction should I move to reduce the Cost?"

But the derivative only gives us the direction. It does **not** move the model. So now the next question is:

Once the computer knows the direction, how does it actually improve itself?

The answer is **Gradient Descent**.

Imagine You Are Lost

Imagine you are standing on a mountain. Your goal is to reach the bottom. Unfortunately, you are wearing a blindfold. You cannot see the mountain. You don't know where the bottom is.

So what do you do?

You take one small step. If the ground goes downward, you continue walking. If the ground goes upward, you turn around. After every step, you check again. Eventually, you reach the bottom. This is exactly what Gradient Descent does.

How Does This Relate to Linear Regression?

Let's compare the two situations.

Mountain	Linear Regression
Mountain	Cost Function
Bottom of the mountain	Minimum Cost
Your position	Current values of θ_0 and θ_1
Walking	Updating θ values
Reaching the bottom	Finding the best model

Gradient Descent simply keeps walking downhill until it reaches the lowest point.

Start With Random Values

The computer does **not** know the correct values of θ_0 and θ_1 . So it starts with random values. For example, $\theta_0 = 0$ $\theta_1 = 0.5$ Could it start with $\theta_1 = 10$?

Yes. Could it start with $\theta_1 = -7$?

Yes. It doesn't matter. Gradient Descent will keep improving those values.

Make Predictions

Using the current values of θ_0 and θ_1 , the model predicts the output. For example,

$$\hat{y} = \theta_0 + \theta_1 x$$

Calculate the Error

Now compare

Actual Value

and Predicted Value. This gives the Error.

Calculate the Cost

Instead of looking at one Error, calculate the Cost Function. Now the model knows

How wrong it is overall.

Calculate the Derivative

Next, calculate the derivative of the Cost Function. Suppose the derivative is -2. What does that mean?

It means

Increasing θ_1 will reduce the Cost.

Now suppose the derivative is +3. That means

Increasing θ_1 will increase the Cost.

So instead, we should decrease θ_1 . Notice something. The derivative is simply giving us directions.

Update θ

Now comes the most famous equation in Gradient Descent.

$$\theta_{new} = \theta_{old} - \alpha \text{ times } \textit{Derivative}$$

Don't memorize it. Let's understand it.

θ_{old}

This is the current value. Suppose $\theta_1 = 0.5$

Derivative

This tells us the direction. Suppose Derivative = -2

α (Alpha)

This is called the **Learning Rate**. It decides

How big should the next step be?

Think of it like walking. You already know which direction to walk. Now you must decide

"Should I take a tiny step or a huge step?"

That decision is made by the Learning Rate.

Why Do We Subtract?

This is one of the most common interview questions. Let's understand it using numbers. Suppose Current $\theta_1 = 0.5$ Learning Rate = 0.1 Derivative = -2. Now update θ .

$$\begin{aligned}\theta_{new} &= 0.5 - 0.1(-2) \\ &= 0.5 + 0.2 = 0.7\end{aligned}$$

Notice what happened. The derivative was negative. Subtracting a negative number automatically increased θ_1 . Exactly what we wanted. Now let's try another example. Suppose Derivative = +2

$$\theta_{new} = 0.5 - 0.1(2) = 0.3$$

Now θ_1 became smaller. Again, that is exactly what we wanted. This is why we always subtract the derivative. The mathematics automatically moves θ in the correct direction.

Understanding the Learning Rate

The Learning Rate controls the size of every step. Let's see what happens in three situations.

Learning Rate Too Small

Suppose

$\alpha = 0.00001$ The model takes tiny steps. Eventually, it will reach the minimum. But it may take thousands of iterations. Training becomes very slow.

Learning Rate Too Large

Suppose

$\alpha = 100$ Now every step is huge. The model jumps over the minimum. Then jumps back. Then jumps again. It keeps overshooting. It may never reach the minimum.

Good Learning Rate

A good Learning Rate finds a balance. The model learns quickly, but without overshooting the minimum.

When Does Gradient Descent Stop?

The model repeats the same process again and again.

The model repeats this cycle: make predictions, calculate error, calculate cost, calculate the derivative, update θ , and repeat.

Eventually, the Cost stops decreasing. The derivative becomes very close to zero. At this point, the model says,

"I can't improve much anymore."

Training stops.

The Entire Training Process

Let's put everything together. Start with random θ values, make predictions, calculate the error, calculate the cost, calculate the derivative, update θ , and repeat. With each useful update, the cost becomes smaller and the model moves closer to the Best Fit Line.

Gradient Descent is an optimization algorithm that repeatedly updates the model parameters in the direction that reduces the Cost Function until the minimum Cost is reached.

Evaluation Metrics

Our model has finished training. It has found the best values of θ_0 and θ_1 . A new question appears.

How do we know if our model is actually good?

Finding the Best Fit Line is not enough. We need a way to measure how well that line performs. That is the job of **Evaluation Metrics**.

Think of them as the report card of your model. Just as a student receives marks after an exam, a machine learning model receives evaluation metrics after making predictions.

Our Example

Suppose our model makes the following predictions.

Actual (y)	Predicted ($\hat{y}$)
10	8
20	23

Actual (y)	Predicted ($\hat{y}$)
30	28
40	45

Let's first calculate the errors.

Actual	Predicted	Error (y - $\hat{y}$)
10	8	2
20	23	-3
30	28	2
40	45	-5

Every evaluation metric starts from these errors. The only difference is **how each metric treats them**.

1. Mean Absolute Error (MAE)

Imagine your manager asks,

“On average, how much is our model wrong?”

That's exactly what MAE answers.

Mean Absolute Error (MAE) is the average of the absolute values of all prediction errors.

Formula

$$MAE = \frac{\sum |y - \hat{y}|}{n}$$

Our errors are

2 -3 2 -5 Take the absolute values. 2 3 2 5 Now find the average.

$$MAE = \frac{2 + 3 + 2 + 5}{4}$$

$$MAE = \frac{12}{4} = 3$$

So,

On average, our model is wrong by 3 units.

Simple.

Why Absolute Value?

Imagine these two errors. +5 -5 If we simply added them, they would become 0. That would incorrectly suggest the model made no mistakes. Absolute value removes the sign, so every mistake counts.

Think of MAE as

A fair teacher.

Every mistake is treated equally. A mistake of 10 is exactly twice as bad as a mistake of 5.

When Should We Use MAE?

Use MAE when

Every mistake has equal importance.

2. Mean Squared Error (MSE)

Now imagine another manager says,

"Small mistakes are okay, but really big mistakes are unacceptable."

MAE cannot do that. Every mistake is treated equally. So we need another metric.

Mean Squared Error (MSE) is the average of the squared prediction errors.

Formula

$$MSE = \frac{\sum (y - \hat{y})^2}{n}$$

Our errors are

2 -3 2 -5 Square every error. 4 9 4 25 Now calculate the average.

$$MSE = \frac{4 + 9 + 4 + 25}{4}$$

$$MSE = \frac{42}{4} = 10.5$$

Why Square the Errors?

Suppose two models make these errors. Model A has errors 1, 2, 3. Model B has errors 1, 2, 10. Without squaring, 10 doesn't seem dramatically larger than 3. After squaring, After squaring, Model A has 1, 4, 9. Model B has 1, 4, 100. Now the huge mistake becomes impossible to ignore. That is exactly why MSE exists.

Think of MSE as

| A strict teacher.

Small mistakes are acceptable. Large mistakes receive a much heavier penalty.

When Should We Use MSE?

Use MSE when

| Large prediction errors are much more costly than small ones.

3. Root Mean Squared Error (RMSE)

MSE solves one problem, but creates another. Suppose we are predicting house prices. House prices are measured in **Dollars**. After squaring, MSE is measured in **Dollars²**. What does "Dollars squared" even mean?

Not much. So we fix it.

RMSE is simply the square root of MSE.

Formula

$$RMSE = \sqrt{MSE}$$

Suppose

MSE = 25 Then

$$RMSE = \sqrt{25} = 5$$

Now the error is back in **Dollars**. Easy to understand.

Think of RMSE as

| The same strict teacher, but reporting marks in normal units.

When Should We Use RMSE?

Use RMSE when

- Large mistakes should receive more punishment.
- You also want the answer in the original unit.

4. R² (R-Squared)

The first three metrics answer

"How wrong is my model?"

R^2 asks a completely different question.

Imagine This

Suppose you don't build any Machine Learning model. Instead, you simply predict the **average value** for every input. For example, suppose the average house price is **300,000**. *Nomatterwhathouse someone shows you, your answer is always 300,000*. This is called a **baseline model**. Now compare that with your Linear Regression model. Which one performs better?

That is exactly what R^2 measures.

R^2 measures how much better your model performs compared to simply predicting the mean for every data point.

Formula

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Don't worry about the formula yet. Let's understand the idea.

Understanding the Formula

SS_{res}

The total error made by **our model**.

SS_{tot}

The total error made by **always predicting the mean**.

The formula simply compares these two. If our model has much smaller errors than the mean model, R^2 becomes large. If both models perform the same, R^2 becomes zero. If our model performs even worse, R^2 becomes negative.

Understanding the Values

$R^2 = 1$

Perfect model. Every prediction is correct.

$R^2 = 0$

Your model is no better than simply predicting the mean.

$$R^2 < 0$$

Your model is actually worse than predicting the mean. This usually means the model is performing very poorly.

Think of R^2 as

"Is my machine learning model actually smarter than just guessing the average?"

When Should We Use R^2 ?

Use R^2 when

You want to measure how well your model explains the data.

5. Adjusted R^2

Now imagine we build two models. Model A Features

- House Size
- Bedrooms

Adjusted $R^2 = 0.82$

Model B

Features

- House Size
- Bedrooms
- Garage

Adjusted $R^2 = 0.86$ Which model is better?

Model B. Why?

Because adding Garage improved the score. Now another example. Model A Adjusted $R^2 = 0.82$ Model B Adjusted $R^2 = 0.80$ Now adding Garage actually reduced the score. That means Garage made the model more complicated, but did not improve it enough.

Adjusted R^2 tells us whether adding a new feature actually improved the model enough to justify the extra complexity.

Formula

$$\text{Adjusted } R^2 = 1 - \frac{(1 - R^2)(n - 1)}{(n - p - 1)}$$

where n = Number of data points p = Number of features

One Important Thing to Remember

A single Adjusted R^2 value tells you very little. You always compare **two models**. That comparison tells you whether the new feature was worth adding. This is one of the most common interview questions.

Think of Adjusted R^2 as

A hiring manager.

Adding another employee increases the team size. But did that employee actually improve the team's performance?

If yes, keep them. If not, the extra complexity wasn't worth it.

How the Idea Comes Together

Metric	What It Measures	Best Used When
MAE	Average prediction error	Every mistake is equally important
MSE	Average squared error	Large mistakes should receive heavier punishment
RMSE	Square root of MSE	Same as MSE, but easier to interpret
R^2	Better than predicting the mean?	Measuring overall model quality
Adjusted R^2	Was the new feature worth adding?	Comparing models with different numbers of features

Multiple Linear Regression

For example, Study Hours can predict Exam Score. House Size can predict House Price. This type of Linear Regression is called **Simple Linear Regression** because it uses only one input variable.

But now let's ask a question.

Is one feature enough to solve real-world problems?

Most of the time, the answer is **No**.

Let's Think About House Prices

Suppose two houses are exactly the same size.

House	Size	Price
A	2000 sq ft	\$350,000
B	2000 sq ft	\$500,000

Both houses have the same size. So why are their prices different?

Because house price depends on much more than size. For example,

- Number of Bedrooms
- Number of Bathrooms
- Location
- Age of the House
- Garage
- Swimming Pool
- School District
- Crime Rate

House Size is only **one piece of the puzzle**. To make better predictions, we need to consider **multiple features**.

What is Multiple Linear Regression?

Multiple Linear Regression is a Linear Regression model that uses two or more input features to predict one output. Instead of asking,

"How does house size affect price?"

it asks,

"How do all these features together affect price?"

The New Equation

Earlier, our equation was

$$\hat{y} = \theta_0 + \theta_1 x$$

Now it becomes

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots + \theta_n x_n$$

At first glance, this looks scary. But don't worry. Nothing has really changed. We simply added more input variables.

Understanding the Equation

Suppose we are predicting house prices.

Feature	Meaning
x_1	House Size
x_2	Bedrooms
x_3	Bathrooms

Feature	Meaning
x_4	House Age

Each feature gets its own coefficient.

Coefficient	Controls
θ_1	House Size
θ_2	Bedrooms
θ_3	Bathrooms
θ_4	House Age

Each coefficient tells us

How much that feature contributes to the final prediction.

A Complete Example

Suppose our equation is

$$\hat{y} = 50000 + 200x_1 + 15000x_2 - 1000x_3$$

where

- x_1 = House Size
- x_2 = Number of Bedrooms
- x_3 = House Age

Now suppose a house has

- House Size = 2000 sq ft
- Bedrooms = 3
- Age = 10 years

Let's calculate the prediction.

$$\begin{aligned}
 \hat{y} &= 50000 + 200(2000) + 15000(3) - 1000(10) \\
 &= 50000 + 400000 + 45000 - 10000 \\
 &= 485000
 \end{aligned}$$

So the predicted price is **\$485,000** Notice something. Each feature contributed its own amount to the final prediction. The computer simply added all those contributions together.

Does the Learning Process Change?

This is the best part. **No.** Everything you learned in the previous ideas still applies. The computer still:

1. Makes Predictions
2. Calculates Error
3. Calculates Cost
4. Finds the Derivative
5. Uses Gradient Descent
6. Updates θ values

The only difference is that instead of updating

- θ_0
- θ_1

it now updates

- θ_0
- θ_1
- θ_2
- θ_3
- ...
- θ_n

The learning process remains exactly the same.

A New Problem Appears

Now imagine we keep adding more and more features. House Size, Bedrooms, Bathrooms, Garage, Swimming Pool, Roof Color, Owner's Favorite Movie, and Owner's Lucky Number Will the predictions always become better?

Probably not. Some features are useful. Some features add no useful information. Some features simply confuse the model. So we need a way to decide

Which features are worth keeping?

Now You Can Finally Understand Adjusted R^2

Remember the previous idea. Adjusted R^2 asks

"Was adding this new feature actually worth it?"

Now it makes complete sense. Suppose Model A uses

- House Size
- Bedrooms

Adjusted $R^2 = 0.84$ Now we add Garage Model B Adjusted $R^2 = 0.88$ Great!

Garage improved the model. Now another example. Model A Adjusted $R^2 = 0.84$ Model B Adjusted $R^2 = 0.82$ The score became worse. That means Garage increased the complexity, but did not improve the prediction enough.

So it wasn't worth adding. This is exactly why Adjusted R^2 exists.

Simple Linear Regression vs Multiple Linear Regression

Simple Linear Regression	Multiple Linear Regression
One input feature	Two or more input features
One coefficient	One coefficient for each feature
Simpler model	More realistic model
Easier to understand	Better for most real-world problems

Simple Linear Regression uses one input feature to predict an output. Multiple Linear Regression uses two or more input features to predict an output.

Underfitting, Overfitting and Regularization

For example, adding

- House Size
- Bedrooms
- Bathrooms

usually gives better predictions than using only House Size. A new question appears.

Should we keep adding more and more features forever?

At first, the answer seems obvious. More information should mean better predictions. Right?

Surprisingly, the answer is **No**. Sometimes adding more information actually makes the model worse. To understand why, let's look at an example.

Imagine Three Students

Three students are preparing for the same exam.

Student A

Student A studies for only 20 minutes. He enters the exam. He doesn't know enough to answer most questions. He performs badly.

Student B

Student B studies every topic carefully. He understands the concepts. He practices many questions. During the exam, even if the teacher asks a new question, he can still solve it.

Student C

Student C does something different. Instead of understanding the concepts, he memorizes last year's question paper. Every answer. Every sentence. Even the spelling mistakes. The next day, the teacher changes the questions. Student C struggles.

Now ask yourself, Which student do you want to be?

Obviously, Student B. Machine Learning works exactly the same way.

Underfitting

Student A represents **Underfitting**.

Underfitting happens when the model is too simple to learn the relationship in the data. It doesn't understand enough.

So it performs poorly even on the training data.

Signs of Underfitting

Training Error: High. Testing Error: High.

Example

Suppose house price depends on

- Size
- Bedrooms
- Location

But your model only uses House Size. The model is missing important information. Predictions will usually be poor.

Overfitting

Student C represents **Overfitting**.

Overfitting happens when the model memorizes the training data instead of learning the real pattern. The model becomes excellent at training data, but performs poorly on new data.

Signs of Overfitting

Training Error: Very Low. Testing Error: High.

Example

Suppose we train our model using House Size, Bedrooms, Bathrooms, Location, Garage, Swimming Pool, Roof Color, House Number, Owner's Favorite Movie, and Owner's Lucky Number. Some of these features have nothing to do with house prices. Instead of learning useful patterns, the model starts learning random noise. It becomes unnecessarily complicated.

Good Fit

Student B represents the ideal model.

A **Good Fit** means the model learned the real relationship in the data. It performs well on both training data and new unseen data.

Signs of a Good Fit

Training Error: Low. Testing Error: Low.

Why Is Overfitting Dangerous?

Remember why we built Machine Learning in the first place. Not to memorize data. But to predict **new data**. If the model only performs well on training data, it is almost useless in the real world.

How Do We Prevent Overfitting?

There are many techniques. One of the most important is called **Regularization**.

What is Regularization?

Imagine you're writing an essay. Your teacher tells you,

"Don't use unnecessary words."

Keep your writing simple. Write only what is important. Regularization tells the model exactly the same thing.

Don't become more complicated than necessary.

How Does Regularization Work?

Remember our equation.

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Each θ tells us how important a feature is. Sometimes, the model gives extremely large values to some θ 's. Very large θ values often indicate that the model is becoming too sensitive to the training data. Regularization adds a penalty for making those θ values too large. As a result, the model becomes simpler and usually performs better on new data.

Ridge Regression (L2 Regularization)

Imagine a football coach. Every player is allowed to stay on the team. But nobody is allowed to become too dominant. Everyone must contribute in a balanced way. That is Ridge Regression.

Ridge Regression reduces large coefficient values but usually does not make them exactly zero.

Lasso Regression (L1 Regularization)

Now imagine another coach. This coach says, "If a player isn't contributing, they leave the team." That is Lasso Regression.

Lasso Regression can reduce some coefficients all the way to zero, effectively removing those features from the model.

Elastic Net

Sometimes, we want the advantages of both Ridge and Lasso. Elastic Net combines both techniques.

Elastic Net combines L1 and L2 regularization to balance coefficient reduction and feature selection.

Comparing Them

Method	What It Does
Ridge	Reduces large coefficients
Lasso	Can reduce coefficients to zero
Elastic Net	Combines Ridge and Lasso

How the Idea Comes Together

Imagine building a team. Underfitting You hired too few people. The work doesn't get done.

Good Fit

You hired the right number of skilled people. Everything works efficiently.

Overfitting

You hired far too many people. Some of them don't even have useful jobs. The team became expensive, confusing, and inefficient. Machine Learning behaves the same way. The goal is not the biggest model. The goal is the **right model**.

Assumptions of Linear Regression

By now, you know almost everything about Linear Regression. You know:

- How it makes predictions.
- How it measures Error.
- How it calculates Cost.
- How it learns using Gradient Descent.
- How it evaluates itself.
- How to avoid Overfitting.

Now imagine your manager comes to you and says,

|"Let's use Linear Regression for this problem."

Should you immediately start building the model?

No. The first question should always be:

|"Is Linear Regression the right algorithm for this data?"

This is where assumptions become important.

What Are Assumptions?

Imagine you buy a car. The manufacturer says,

|"This car performs best on normal roads."

Can you still drive it on sand?

Yes. Can you drive it through a river?

Maybe. But don't expect the same performance. The car isn't bad. You're simply using it in a situation it wasn't designed for. Linear Regression is the same. It performs best when certain conditions are true. Those conditions are called **assumptions**.

Assumptions are conditions that should be reasonably true for Linear Regression to produce reliable results. Notice something. I didn't say

|"They must always be true."

In real life, very few datasets satisfy every assumption perfectly. The goal is simply to be **reasonably close**.

Assumption 1 – The Relationship Should Be Linear

This is the most important assumption. Linear Regression tries to fit a **straight line**. So naturally, the relationship between the input and output should be approximately a straight line.

Example

Suppose we have this data.

Study Hours	Marks
1	50
2	60
3	70
4	80

Every extra hour increases the marks by about the same amount. A straight line fits nicely. Linear Regression works well.

Now imagine this.

Speed	Fuel Consumption
20	5
40	8
60	15
80	30

Notice how the increase becomes larger and larger. The relationship is curved. A straight line cannot describe it very well. Linear Regression may struggle.

Assumption 2 – The Data Should Be Independent

Imagine you're predicting house prices. Each house is different. Each row represents a different property. Perfect. Now imagine predicting today's stock price. Today's stock price depends heavily on yesterday's stock price. The observations are connected. They are **not independent**. Linear Regression doesn't naturally handle this type of data very well.

Assumption 3 – The Error Should Be Consistent

This assumption has a long statistical name: **Homoscedasticity**. Don't worry about remembering the word. Remember the idea.

Imagine your model predicts house prices. For small houses, the prediction error is around $\pm 5,000$. *For medium houses, the error is also around $\pm 6,000$.* For large houses, the error is around $\pm \$4,000$. The errors stay roughly the same size. This is good.

Now imagine another situation. Small houses Error = $\pm 3,000$ *Luxury houses Error* = $\pm 250,000$. Now the errors become larger and larger as the price increases. This violates the assumption.

Assumption 4 – Features Should Not Be Duplicates

Imagine you're predicting house prices using these features.

- House Size (Square Feet)
- House Size (Square Meters)

Are these really different features?

No. They contain exactly the same information. The model becomes confused because both features explain the same thing. This problem is called **Multicollinearity**.

Multicollinearity happens when two or more features contain almost the same information.

Assumption 5 – The Errors Should Look Random

After training the model, look at the errors. If the errors are scattered randomly, that's a good sign. If the errors form a clear pattern, it usually means the model has missed something important.

Example

Suppose every expensive house is always underpriced by the model. That pattern suggests the model hasn't fully captured the relationship. A better model or additional features may be needed.

Which Assumption Is the Most Important?

If you remember only one assumption, remember this one:

■ **The relationship should be approximately linear.**

Why?

Because the entire algorithm is trying to fit a straight line. If the true relationship is highly curved, Linear Regression starts with a disadvantage.

How the Idea Comes Together

Assumption	Simple Meaning
Linearity	The relationship should look like a straight line.
Independence	Each row should be independent of the others.
Consistent Error	Errors should stay roughly the same size.
No Multicollinearity	Features should not contain duplicate information.
Random Errors	Errors should not follow a pattern.

Final Thoughts

You have now learned Linear Regression from the ground up. You didn't just memorize formulas. You learned:

- Why every formula exists.
- What problem each concept solves.
- How all the concepts connect together.

This is exactly how Machine Learning is used in the real world. Every algorithm begins with a problem. Every formula is created to solve that problem. When you understand the **problem first**, the formulas become much easier to remember.

What's Next?